

Assignment 1

Min Zhu

April 16, 2026

1 Overall work

In this assignment, I have successfully trained a one-layer network to classify images from the CIFAR-10 dataset. The model is trained using mini-batch gradient descent. It adopts the softmax function as its activation function and the training process is guided by the cross-entropy loss. Through this basic linear model, the best accuracy achieved on the test dataset is 39.41%.

2 Implementation of gradient computation

In Step 7 of the assignment (the BackwardPass part), I implemented the analytical computation of gradients and verified their correctness by comparing with the results via `Pytorch`.

To validate the implementation, I conducted gradient checking on a small subset of the training data. Specifically, I compared the gradients produced from my analytically computation with those obtained from PyTorch's built-in backward function. Both absolute and relative differences were evaluated. The maximum absolute error and relative error observed during the test are shown as an example in Table 1.

The results show that the discrepancies between the two sets of gradients are consistently below the predefined threshold of $\epsilon = 1 \times 10^{-6}$, indicating that the analytical gradients are computed correctly. Based on this verification, I proceeded to use the implemented gradient computation on the full training dataset.

Table 1: Verification of gradients

Parameter	Max abs error	Max relative error
W	4.16e-17	2.74e-14
b	2.49e-17	7.24e-14

3 Results of different parameter settings

After constructing the baseline network, different parameter settings are adopted to evaluate their impact on model performance. Here, the settings for times of epochs and batch sizes are fixed and same, being $n_epochs = 40$ and $n_batch = 100$ separately.

3.1 Graph of lost and cost function

First, the graphs of the lost and the cost function on both the training data and the validation data under four different settings of parameters are shown in Fig 1.

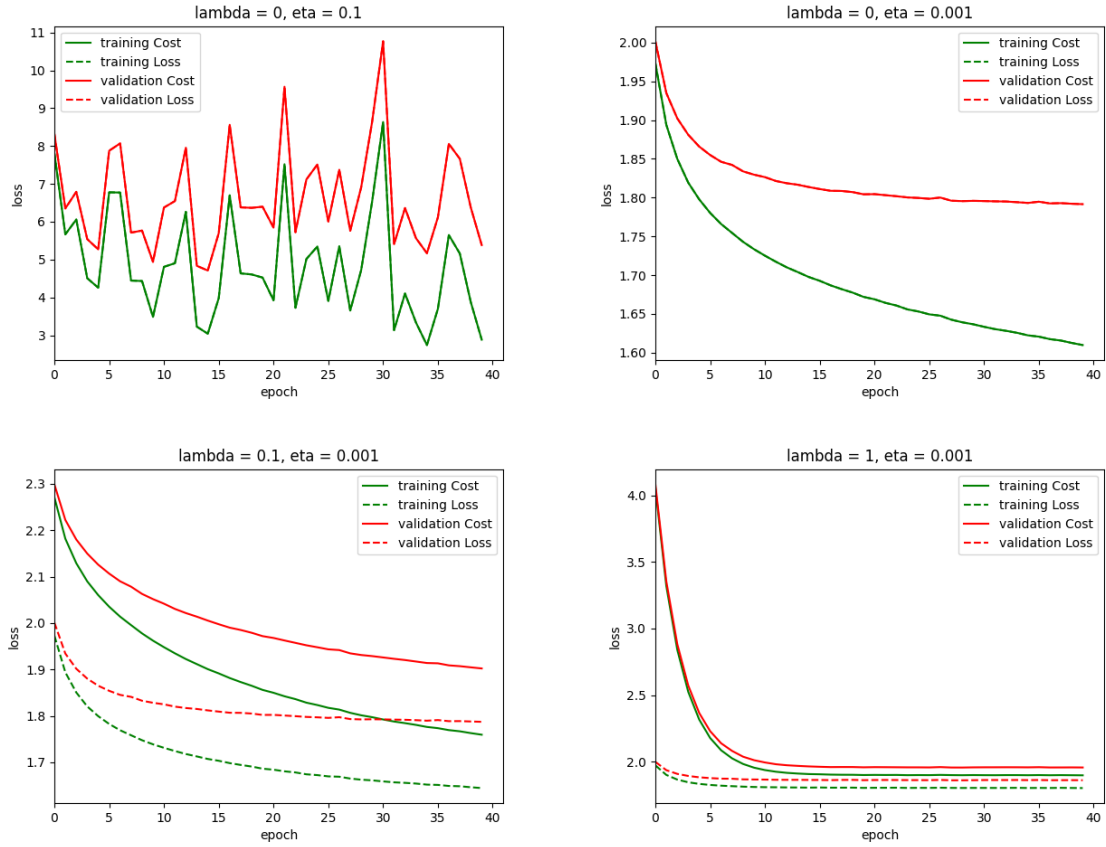


Figure 1: Lost and Cost

3.2 Image of learnt weight matrix

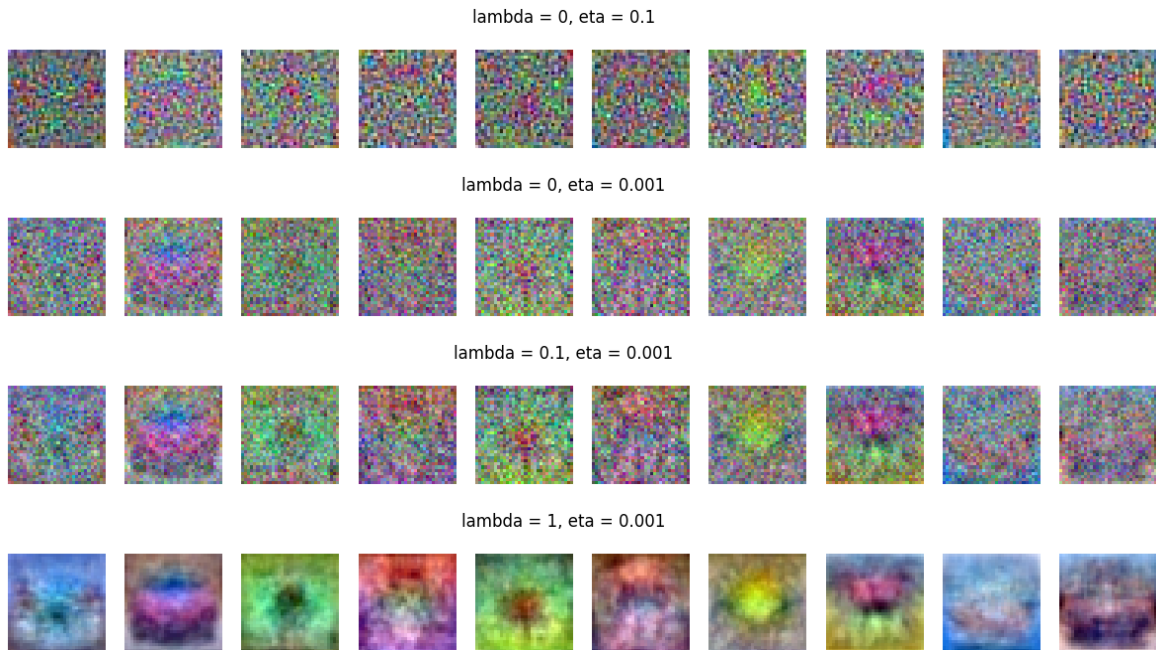


Figure 2: Image of learnt weight matrix

Fig 2 demonstrates the visualization of the weight matrix W after the completion of training under different settings.

3.3 Comparison of test accuracy

The final accuracy computed on the test data is as following:

Table 2: Accuracy on test data

Setting ID	Lam	Eta	Accuracy
1	0	0.1	29.75%
2	0	0.001	39.41%
3	0.1	0.001	39.24%
4	1	0.001	37.09%

4 Effect of parameters

4.1 Increase of regularization

Comparing Settings 2, 3, and 4, where the regularization parameter λ increases from 0 to 0.1 and 1 while keeping other parameters constant, we can observe the increasing impact of regularization on the model’s performance.

As illustrated in Fig1, increasing λ effectively narrows the generalization gap between the training and validation cost curves, indicating a reduction in overfitting. However, this improved generalization comes at the expense of higher overall error: after 40 epochs, the converged validation cost increases progressively (from approximately 1.8 to 1.95, and finally to 2.0).

Furthermore, Figure 2 reveals a clear transition in the learned features as λ increases. At low regularization strengths ($\lambda = 0$), the weight templates are dominated by stochastic noise, making it hard to capture the item shape. As λ increases to 0.1 and 1.0, the weight matrices become smoother and more interpretable with item structures gradually emerging. This transformation occurs because increasing regularization term forces the model to disregard localized noise and prioritize global, structural patterns. However, the test accuracy exhibits a downward trend as λ grows.

All these observations demonstrate that while regularization improves the generalization ability of a model, it also leads to an increase in model bias due to excessive constraints.

4.2 Correct learning rate

Settings 1 and 2 are compared to evaluate the effect of the learning rate, as the only difference between them is $\eta = 0.1$ in Setting 1 and $\eta = 0.001$ in Setting 2.

When $\eta = 0.1$, the validation cost exhibits stochastic oscillations and fails to demonstrate a convergence trend. The weight matrix is disordered, making it difficult to capture meaningful patterns and the test accuracy remains low (approximately 29.75%). All these observations indicate inability to extract meaningful representative features and inadequate learning. The big step size causes the optimizer to miss the potential local minima. Conversely, decreasing the learning rate to $\eta = 0.001$ yields a monotonic and smooth validation cost curve with a clear trend toward convergence, also leads to a substantial improvement of around 10% in test accuracy. The great difference between these two settings highlights that the learning rate has a substantial impact on the model’s learning capability.